DATABASE MANAGEMENT SYSTEM

Unit-III: Relational Database Design

Comprehensive Study Notes with Normalization Concepts

UNIT-III: RELATIONAL DATABASE DESIGN

Topic 1: Relational Schema and Relational Design

1.1 What is a Relational Schema?

A **relational schema** is the logical description of the database structure. It defines the tables, their attributes, and the relationships between them.

```
╔══════════════════════════════════════════════════════════════════╗
║                    RELATIONAL SCHEMA EXAMPLE                    ║
╚══════════════════════════════════════════════════════════════════╝

STUDENT(StudentID, Name, Age, DeptID)
DEPARTMENT(DeptID, DeptName, Location)
COURSE(CourseID, CourseName, Credits, DeptID)
ENROLLMENT(StudentID, CourseID, Grade)

Primary keys are underlined: StudentID, DeptID, CourseID
Foreign keys:
  DeptID in STUDENT references DEPARTMENT(DeptID)
  DeptID in COURSE references DEPARTMENT(DeptID)
  StudentID in ENROLLMENT references STUDENT(StudentID)
  CourseID in ENROLLMENT references COURSE(CourseID)
```

1.2 Relational Database Design

Relational Database Design is the process of creating a database schema that is efficient, consistent, and avoids data anomalies.

Goal	Description
No Redundancy	Avoid duplicate data
No Anomalies	Avoid insertion, deletion, and update anomalies
Data Integrity	Ensure data accuracy and consistency
Performance	Efficient query processing
Flexibility	Easy to adapt to changing requirements

1.3 Design Process Steps

```
╔══════════════════════════════════════════════════════════════════╗
║                    DATABASE DESIGN PROCESS                       ║
╚══════════════════════════════════════════════════════════════════╝

1. Requirements Analysis
        |
2. Conceptual Design (E-R Model)
        |
3. Logical Design (Relational Schema)
        |
4. Normalization
        |
5. Physical Design (Storage, Indexes)
```

Topic 2: Functional Dependency

2.1 What is Functional Dependency?

A **functional dependency** is a constraint between two sets of attributes in a relation. It specifies that the value of one set of attributes determines the value of another set.

```
╔══════════════════════════════════════════════════════════════════════╗
║                      FUNCTIONAL DEPENDENCY                          ║
╚══════════════════════════════════════════════════════════════════════╝

X → Y means: For each value of X, there is exactly one value of Y

Example: StudentID → Name, Age, DeptID
(StudentID uniquely determines Name, Age, and DeptID)

┌────────────────────────────────────────────────────────────┐
│ StudentID │ Name     │ Age │ DeptID │
├────────────────────────────────────────────────────────────┤
│ S001      │ Rahul    │ 20  │ D01    │
├────────────────────────────────────────────────────────────┤
│ S002      │ Priya    │ 21  │ D02    │
└────────────────────────────────────────────────────────────┘
For StudentID S001 → Name is Rahul (unique)
For StudentID S001 → Age is 20 (unique)
```

2.2 Types of Functional Dependencies

Type	Description	Example
Trivial FD	X → Y where Y is subset of X	StudentID, Name → StudentID
Non-Trivial FD	X → Y where Y is not subset of X	StudentID → Name
Full FD	X → Y and no subset of X determines Y	StudentID → Name
Partial FD	X → Y and some subset of X determines Y	StudentID, CourseID → Name (Name depends on StudentID only)

Transitive FD	X → Y and Y → Z implies X → Z	StudentID → DeptID → DeptName

2.3 Armstrong's Axioms

Armstrong's Axioms are the fundamental inference rules for functional dependencies:

Axiom	Description	Formula
Reflexivity	If Y is subset of X, then X → Y	If Y ⊆ X, then X → Y
Augmentation	If X → Y, then XZ → YZ	X → Y ⇒ XZ → YZ
Transitivity	If X → Y and Y → Z, then X → Z	X → Y, Y → Z ⇒ X → Z

Additional Rules:

Rule	Description	Formula
Union	If X → Y and X → Z, then X → YZ	X → Y, X → Z ⇒ X → YZ
Decomposition	If X → YZ, then X → Y and X → Z	X → YZ ⇒ X → Y, X → Z

2.4 Closure of Attribute Set

Closure of an attribute set X (denoted as X+) is the set of all attributes that can be functionally determined by X.

Algorithm:

```
1. Set X+ = X
2. For each functional dependency (A → B):
   If A ⊆ X+, then add B to X+
3. Repeat step 2 until no more attributes can be added
```

Example:

```
Given: Relation R(A, B, C, D, E)
FDs: A → B, B → C, AC → D

Find A+:
```

```
Step 1: A+ = {A}
Step 2: A → B ⇒ A+ = {A, B}
Step 3: B → C ⇒ A+ = {A, B, C}
Step 4: AC → D ⇒ C is in A+ ⇒ A+ = {A, B, C, D}
Result: A+ = {A, B, C, D}
```

2.5 Super Key and Candidate Key using Closure

Key Type	Definition	Using Closure
Super Key	Set of attributes that uniquely identifies a tuple	X+ contains all attributes
Candidate Key	Minimal super key	X is super key and no proper subset is super key

Example 1: Find Candidate Keys

```
Given: R(A, B, C, D, E) with FDs: A → B, B → C, C → D, D → E

Solution:
Find closures:
- A+ = {A, B, C, D, E} = All attributes ⇒ A is a Candidate Key
- B+ = {B, C, D, E} (A missing) ⇒ Not Candidate Key
- C+ = {C, D, E} (A,B missing) ⇒ Not Candidate Key
- D+ = {D, E} ⇒ Not Candidate Key
- E+ = {E} ⇒ Not Candidate Key

Result: Candidate Key = {A}
```

Example 2: Find Candidate Keys

```
Given: R(A, B, C, D) with FDs: AB → C, C → D

Solution:
Find closures:
- AB+ = {A, B, C, D} = All attributes ⇒ AB is a Super Key
- A+ = {A} ⇒ Not Candidate Key
- B+ = {B} ⇒ Not Candidate Key
- C+ = {C, D} ⇒ Not Candidate Key
```

```
- AC+ = {A, C, D} (B missing) ⇒ Not Candidate Key
- BC+ = {B, C, D} (A missing) ⇒ Not Candidate Key
- AB is a Candidate Key (no proper subset is super key)

Result: Candidate Key = {A, B}
```

Topic 3: Normal Forms

3.1 What is Normalization?

Normalization is the process of organizing data to reduce redundancy and improve data integrity. It involves decomposing tables to eliminate data anomalies.

```
╔══════════════════════════════════════════════════════════════════╗
║                    NORMALIZATION HIERARCHY                       ║
╚══════════════════════════════════════════════════════════════════╝

        ┌──────────────┐
        │ 1NF (First)  │
        └──────┬───────┘
        ┌──────┴───────┐
        │ 2NF (Second) │
        └──────┬───────┘
        ┌──────┴───────┐
        │ 3NF (Third)  │
        └──────┬───────┘
        ┌──────┴───────┐
        │ BCNF         │
        └──────────────┘
```

3.2 First Normal Form (1NF)

Definition: A relation is in 1NF if all attributes contain **atomic (indivisible) values** and there are no repeating groups.

```
╔══════════════════════════════════════════════════════════════════╗
║                         1NF EXAMPLE                              ║
╚══════════════════════════════════════════════════════════════════╝

NOT IN 1NF:
┌───────────┬─────────────┐
│ StudentID │ Courses     │  ← Repeating group
├───────────┼─────────────┤
│ S001      │ C101,C102  │
└───────────┴─────────────┘

IN 1NF (Atomic values):
┌───────────┬──────┐
│ StudentID │Course│
├───────────┼──────┤
│ S001      │ C101 │
│ S001      │ C102 │
```

```
│ S001        │ C103 │
└──────────────────────┘
```

Rules for 1NF:

1. Each cell has a single value
2. Each column has a unique name
3. All entries in a column are of the same data type

3.3 Second Normal Form (2NF)

Definition: A relation is in 2NF if:

1. It is in 1NF
2. **No partial dependency** exists (no non-prime attribute depends on a subset of a candidate key)

```
╔══════════════════════════════════════════════════════════════════╗
║                         2NF EXAMPLE                              ║
╚══════════════════════════════════════════════════════════════════╝

NOT IN 2NF (Partial Dependency):
Table: ENROLLMENT(StudentID, CourseID, StudentName, Grade)
Candidate Key: {StudentID, CourseID}
Partial Dependency: StudentName → StudentID
(StudentName depends on StudentID only)

SOLUTION: Split into two tables:
  STUDENT(StudentID, StudentName)      ← 2NF
  ENROLLMENT(StudentID, CourseID, Grade) ← 2NF
```

3.4 Third Normal Form (3NF)

Definition: A relation is in 3NF if:

1. It is in 2NF
2. **No transitive dependency** exists (no non-prime attribute depends on another non-prime attribute)

```
╔══════════════════════════════════════════════════════════════════╗
║                         3NF EXAMPLE                              ║
╚══════════════════════════════════════════════════════════════════╝

NOT IN 3NF (Transitive Dependency):
Table: STUDENT(StudentID, Name, DeptID, DeptName)
Functional Dependencies:
```

```
  StudentID → DeptID
  DeptID → DeptName
Transitive Dependency: StudentID → DeptID → DeptName

SOLUTION: Split into two tables:
  STUDENT(StudentID, Name, DeptID)     ← 3NF
  DEPARTMENT(DeptID, DeptName)         ← 3NF
```

3.5 Boyce-Codd Normal Form (BCNF)

Definition: A relation is in BCNF if:

1. It is in 3NF
2. For every functional dependency X → Y, X must be a **super key**

```
╔══════════════════════════════════════════════════════════════════╗
║                          BCNF EXAMPLE                            ║
╚══════════════════════════════════════════════════════════════════╝

NOT IN BCNF:
Table: COURSE_INSTRUCTOR(CourseID, Instructor, Office)
Functional Dependencies:
  CourseID → Instructor
  Instructor → Office
Here, Instructor → Office where Instructor is NOT a super key

SOLUTION: Split into two tables:
  COURSE(CourseID, Instructor)
  INSTRUCTOR(Instructor, Office)
```

3.6 Normal Forms Summary

Normal Form	Condition	Violation
1NF	Atomic values	Repeating groups
2NF	1NF + No partial dependency	Partial dependency
3NF	2NF + No transitive dependency	Transitive dependency
BCNF	3NF + Every determinant is a super key	Non-key determinant

Topic 4: Decomposition

4.1 What is Decomposition?

Decomposition is the process of breaking down a large table into smaller tables to remove redundancy and anomalies.

```
╔══════════════════════════════════════════════════════════════════════╗
║                          DECOMPOSITION                              ║
╚══════════════════════════════════════════════════════════════════════╝

Original Table:
┌─────────────────────────────────────────────────────┐
│ StudentID │ Name  │ DeptID │ DeptName               │
├─────────────────────────────────────────────────────┤
│ S001      │ Rahul │ D01    │ Computer               │
│ S002      │ Priya │ D02    │ Math                   │
│ S003      │ Amit  │ D01    │ Computer               │
└─────────────────────────────────────────────────────┘

Decomposed Tables:

STUDENT:
┌─────────────────────────────┐
│ StudentID │ Name  │ DeptID │
├─────────────────────────────┤
│ S001      │ Rahul │ D01    │
│ S002      │ Priya │ D02    │
│ S003      │ Amit  │ D01    │
└─────────────────────────────┘

DEPARTMENT:
┌──────────────────┐
│ DeptID │ DeptName│
├──────────────────┤
│ D01    │ Computer│
│ D02    │ Math    │
└──────────────────┘
```

4.2 Types of Decomposition

Type	Description
Lossless Decomposition	No information is lost; can join back to original
Lossy Decomposition	Some information is lost; cannot join back

Dependency Preservation	All functional dependencies are preserved

4.3 Lossless Join Decomposition

A decomposition is **lossless** if the natural join of the decomposed tables gives back the original table.

Condition: For R decomposed into R1 and R2, the decomposition is lossless if:

```
R1 ∩ R2 → R1   OR   R1 ∩ R2 → R2
```

Topic 5: Dependency Preservation

5.1 What is Dependency Preservation?

Dependency preservation ensures that all functional dependencies that hold in the original relation are also enforced in the decomposed relations.

Example:

```
Original: R(A, B, C) with FDs: A → B, B → C
Decomposed: R1(A, B), R2(B, C)
FDs in R1: A → B (Preserved)
FDs in R2: B → C (Preserved)
All FDs are preserved!
```

Topic 6: Multivalued Dependency

6.1 What is Multivalued Dependency?

A **multivalued dependency (MVD)** occurs when two independent attributes are associated with a single primary key.

```
╔══════════════════════════════════════════════════════════════════╗
║                     MULTIVALUED DEPENDENCY                      ║
╚══════════════════════════════════════════════════════════════════╝

X →→ Y (X multi-determines Y)
Means: For each X value, there is a set of Y values independent of Z

Example:
  StudentID →→ CourseID
  StudentID →→ Hobby

Table: STUDENT_HOBBIES(StudentID, CourseID, Hobby)
S001 takes both C101 and C102, and has hobbies Reading and Music
```

6.2 4NF (Fourth Normal Form)

Definition: A relation is in 4NF if:

1. It is in BCNF
2. There are **no multivalued dependencies** that are not also functional dependencies

Example of Violation:

```
Table: STUDENT_COURSE_HOBBY(StudentID, CourseID, Hobby)
MVDs: StudentID →→ CourseID
      StudentID →→ Hobby

SOLUTION: Split into two tables
  STUDENT_COURSE(StudentID, CourseID)
  STUDENT_HOBBY(StudentID, Hobby)
```

Topic 7: HPU Important Questions

Short Answer Questions (2-5 Marks)

1. What is a functional dependency?
2. What is Armstrong's axiom?
3. What is normalization?
4. What is 1NF? Explain.
5. What is 2NF? Explain.
6. What is 3NF? Explain.
7. What is BCNF? Explain.
8. What is a multivalued dependency?
9. What is decomposition?
10. What is dependency preservation?

Long Answer Questions (10 Marks)

1. Explain functional dependency and Armstrong's axioms.
2. Explain normalization and all normal forms.
3. Explain 1NF, 2NF, 3NF with examples.
4. Explain BCNF with examples.
5. Explain decomposition and dependency preservation.
6. Explain multivalued dependency and 4NF.

Objective Questions (1 Mark)

1. A functional dependency X → Y means:
   (a) X determines Y (b) Y determines X (c) Both (d) None
   Answer: (a)
2. 1NF requires:
   (a) Atomic values (b) No partial dependencies (c) No transitive

dependencies (d) None

Answer: (a)

3. 2NF removes:

   (a) Atomic values (b) Partial dependencies (c) Transitive dependencies (d) Multivalued dependencies

   Answer: (b)

4. 3NF removes:

   (a) Atomic values (b) Partial dependencies (c) Transitive dependencies (d) Multivalued dependencies

   Answer: (c)

5. BCNF requires:

   (a) Atomic values (b) No partial dependencies (c) Every determinant is a super key (d) None

   Answer: (c)

Quick Revision

Normal Forms Summary

Normal Form	Condition	Example Violation
1NF	Atomic values	Repeating groups
2NF	1NF + No partial dependencies	StudentName depends on StudentID only
3NF	2NF + No transitive dependencies	DeptName depends on DeptID
BCNF	3NF + Every determinant is a super key	Instructor → Office

Key Formulas

Formula	Description
$X \rightarrow Y$	Functional Dependency
$X+$	Closure of X
$X \rightarrow\rightarrow Y$	Multivalued Dependency
$R1 \cap R2 \rightarrow R1$ or $R2$	Lossless Join Condition

FD Rules

Rule	Formula
Reflexivity	If $Y \subseteq X$, then $X \rightarrow Y$
Augmentation	$X \rightarrow Y \Rightarrow XZ \rightarrow YZ$

Transitivity	$X \rightarrow Y, Y \rightarrow Z \Rightarrow X \rightarrow Z$
Union	$X \rightarrow Y, X \rightarrow Z \Rightarrow X \rightarrow YZ$
Decomposition	$X \rightarrow YZ \Rightarrow X \rightarrow Y, X \rightarrow Z$

End of Unit-III Notes — DATABASE MANAGEMENT SYSTEM — Relational Database Design